

P2537R0

Relax `va_start` Requirements to Match C

Published Proposal, 2022-02-15

Author:

[JeanHeyd Meneide \(https://thephd.dev\)](https://thephd.dev)

Audience:

SG22, LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++26

Latest:

https://thephd.dev/_vendor/future_cxx/papers/d2513.html

Abstract

At the WG14 January/February Meeting on February 1st, WG14 voted heavily in favor of allowing `va_start/arg/end/list`-style functions to be able to take 0 arguments, alongside removing the requirement of a special "last parameter" being passed to `va_start`. C++ has always allowed function declarations and definitions with zero arguments, but intentionally left `va_start` unupdated to match C's existing practice. This paper follows the new direction for C to remove the restrictions from `va_start` in C.

Table of Contents

- 1 Revision History**
 - 1.1 Revision 0 - February 15th, 2022
- 2 Introduction and Motivation**
 - 2.1 The Solution

- 2.1.1 What if the ABI is different?
- 2.1.2 Allow Declaration, not but don't allow Definitions?

3 Specification

- 3.1 Library Wording
 - 3.1.1 Modify Header Synopsis [cstdarg.syn]

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0 - February 15th, 2022

- SG22 saw this paper on February 11th, 2022 and voted in favor of forwarding it towards WG21 and for WG14. They do not see any compatibility issues, but recommended two things:
 - Allowing declarations but not definitions in the C paper;
 - and, talking about ABI changes.
- A section
- Initial Release! 🎉

§ 2. Introduction and Motivation

C merged a paper to remove functions without prototypes ("K&R Functions") from C23, closing the chapter on a nearly 40 year old deprecated feature. Unfortunately, K&R declarations served a number of useful purposes that could not be effectively approximated by any other function declaration in C. In C++, old-style K&R functions on both an ABI and an API level could be approximated with a declaration such as `void f(...);`. This was impossible in C thanks to the requirement in both the language that there must be at least one parameter before the `...`, and that the last parameter must be passed to `va_start`. This is bad because many inter-language function calls and similar were enabled

in plain, standard C by using K&R declarations and then having the non-C side use assembly or other tricks to appropriately handle both passed-in arguments as well as return values:

```
// Pre-C23 K&R Declaration
double compute_values();

int main () {
    // C: allowed usage under K&R rules, removed in C23
    // C++: ill-formed (constraint violation), function takes 0 arguments
    double x = compute_values("with_pi", 2.4, 2.7, true);
    return (int)x;
}
```

The implementation of `compute_values` here could be done VIA assembly or with other tricks in other languages, allowing a C codebase to talk to other programming languages and tools efficiently without having to create a dedicated Foreign Function Interface. Unfortunately, the removal of K&R declarations in C has made the above code illegal in standard C, and taken away a fairly valid use case for prototype-less functions.

§ 2.1. The Solution

C is moving to fix this problem by allowing `void f(...);` as a valid function declaration, as shown in [n2919]. This will allow the same level of power as K&R declarations without the problematic no-arguments-but-takes-any-number-of-arguments-actually double-meaning syntax. In order to do this, C is updating the `va_start` macro as well as fixing its language rules. C++ does not need to fix any of its language rules:

```
// Valid in C23, always valid C++
double compute_values(...);

int main () {
    // C and C++: allowed and portable in both languages
    double x = compute_values("with_pi", 2.4, 2.7, true);
    return (int)x;
}
```

Since C++ includes the `<stdarg.h>` header as part of its library offerings in `<cstdarg>`, some adjustments must be made to the contents of the synopsis and wording for `<cstdarg>` to match the

changes that will be made for C. Additionally, some modifications must be made to the constraints to match the changes C makes. Because this is something that was not previously allowed before, it has no impact on existing implementations and for all major compilers (GCC, MSVC, Clang, and their derivatives) they have the necessary built-in compiler magic to produce working library implementations that do not require the first argument to `va_start`.

An example proving that this is possible is publicly available here:

[ztd.vargs \(https://ztdvargs.readthedocs.io/en/latest/\)](https://ztdvargs.readthedocs.io/en/latest/). The ABI for variadic arguments versus K&R prototypes is not affected because the C ABI did not allow this declaration before, so there is no existing standard code for C that could rely on this function call. C++ may have an ABI for it, but no standards-compliant code could access any of the function arguments using `va_start/va_arg/va_end` thanks to the macro's specification. Therefore, this feature either introduces a new ABI that did not previously exist on the platform at all, or simply utilizes an existing ABI (the example library implementation leverages well-defined existing ABIs in C++ implementations in order to work properly).

§ 2.1.1. What if the ABI is different?

There are a handful of C compilers that allow declarations using variable arguments without a first parameter, and some shared C and C++ compiler implementations which also allow it by having `extern "C"` functions declared without the first parameter in C++. Therefore, there can be compilers which do not share an ABI for their K&R and their variable argument declarations.

The solution here is not something that can be specified in the standard. This paper can only recommend that implementations which need more explicit control over the resulting ABI of their K&R functions may need to provide an attribute, like below, when performing the migration:

```
[[impl::krdecl]] double compute_values(...);

int main () {
    // compute_values uses right register and stack allocation convention
    double x = compute_values("with_pi", 2.4, 2.7, true);
    return (int)x;
}
```

This can alleviate much of the trouble of porting, and can still be automated when upgrading to C23. We do not have any means in the standard to provide an `[[impl::krdecl]]` or similar because ABI, register usage, and similar calling convention work is somewhat outside the scope of the standard.

§ 2.1.2. Allow Declaration, not but don't allow Definitions?

There was discussion about allowing this to change only the ability in C to declare these C functions, and not to define them. This is different from C++ where it is both allowed to be declared and defined with no first parameter. Clang also has an extension that allows these functions to be both declared and defined with C or C++-style name mangling:

```
void func(...) __attribute__((overloadable)) {  
  
}  
  
int main() {  
    func(1, 2, 3);  
}
```

The original paper in [\[n2919\]](#) and this paper posit that it would be too inconsistent to allow declarations (like C++) but disallow definitions (unlike C++) in C. Therefore, the C paper is going forward with changing [va_start](#) and allowing both declarations and definitions. This paper is also being put in the C++ mailing list to bring C++ up to the same level of compatibility with C, if the N2919 change is made. If the change is withdrawn then this paper will be removed.

§ 3. Specification

The specification is relative to the latest C++ Working Draft, [\[n4901\]](#).

§ 3.1. Library Wording

§ 3.1.1. Modify Header Synopsis [[cstdarg.syn](#)]

17.13.2 Header <cstdarg> Synopsis

[[cstdarg.syn](#)]

```
namespace std {  
    using va_list = see below;  
}  
  
#define va_arg(V, P) see below  
#define va_copy(VDST, VSRC) see below  
#define va_end(V) see below  
  
#define va_start(V, P) see below  
#define va_start(V, ...) see below  
  
}
```

- 1 The contents of the header <cstdarg> are the same as the C standard library header <stdarg.h>, with the following changes: The restrictions that ISO C places on the second parameter ,if provided, to the `va_start` macro in header <stdarg.h> are different in this document. The parameter `parmN` is the rightmost parameter in the variable parameter list of the function definition (the one just before the `...`)²⁰⁷. If the parameter `parmN` is a pack expansion ([temp.variadic]) or an entity resulting from a lambda capture ([expr.prim.lambda]), the program is ill-formed, no diagnostic required. If the parameter `parmN` is of a reference type, or of a type that is not compatible with the type that results when passing an argument for which there is no parameter, the behavior is undefined.

SEE ALSO: ISO C 7.16.1.1

§ References

§ Informative References

[N2919]

Alex Gilding; JeanHeyd Meneide. *Relax requirements for variadic parameter lists*. January 25th, 2022. URL: <http://open-std.org/JTC1/SC22/WG14/www/docs/n2919.pdf>

[N4901]

Thomas Köppe. *[Working Draft, Standard for Programming Language C++](#)*. 22 October 2021.

URL: <https://wg21.link/n4901>

[ZTD.VARGS]

JeanHeyd Meneide; Shepherd's Oasis, LLC. *[ztd.vargs](#)*. November 22nd, 2021. URL:

<https://ztdvargs.readthedocs.io/en/latest/>